



Una arquitectura más limpia donde la fuente de la verdad está en la base de datos

Este documento proporciona una descripción detallada de la conjunto de pruebas, diseñado para ayudar a los agentes de codificación a comprender la arquitectura, las estrategias de simulación y la cobertura funcional de las pruebas del entorno local.jeffmeridianlocal_sanity.spec.ts

Description general

El es un conjunto de pruebas integral basado en Playwright que valida el entorno de desarrollo local para Writer Studio y su sitio web asociado generado por Elevenly. Sirve como verificación de la cordura de la interfaz de usuario y como verificación de la integración de herramientas de backend.jeffmeridianlocal_sanity.spec.ts

Objetivo principal: http://localhost:8080

Arquitectura central

Notas de mantenimiento para agentes

- **Agregar nuevos complementos:** Si agrega un nuevo complemento a anti/plugin/, asegúrese de que admita el variable de entorno para mantener la compatibilidad con esta suite.TESTDBFILE
- **Cambios de esquema:** Si el esquema de la base de datos evoluciona, actualice el sql cadena en el nivel superior test.beforeAll bloque para incluir nuevas tablas o columnas.
- **Limpieza:** La suite desvincula automáticamente el archivos después de la ejecución. No almacene datos persistentes en estos archivos.testmock*.db

Prerrequisitos

Para ejecutar estas pruebas con éxito, lo siguiente debe estar disponible en el sistema:

1. **Node.js y dramaturgo:** `npx playwright test`
1. **Python 3:** Para ejecutar los complementos y `plugin_host.py`.
1. **CLI de SQLite3:** Debe estar instalado y disponible en el `$PATH` para la siembra simulada de bases de datos.
1. **Servidor local:** El servidor de desarrollo Eleventy debería estar ejecutándose en `http://localhost:8080`.

Ejecutando las pruebas

Para ejecutar la suite completa:

```
npx playwright test transcripts/tests/
jeffmeridianlocal_sanity.spec.ts
```

Para ejecutar únicamente las pruebas de integración de la herramienta Python:

```
npx playwright test -g "Anti Plugin Tools|
Writer Studio Tools API"
```

1. Estrategia simulada (aislamiento SQLite)

Para garantizar que las pruebas no sean destructivas e idempotentes, la suite utiliza **Aislamiento a nivel de base de datos**.

- **Bases de datos específicas del trabajador:** Al inicio de la prueba (`test.beforeAll`), cada trabajador de Playwright genera un archivo SQLite único (por ejemplo, `.testmock0.db`)
- **Siembra:** La base de datos simulada se siembra utilizando el `sqlite3` CLI con un conjunto estandarizado de datos:
- `A documents` tabla con un “Documental simulado” (ID 999).
- `An appstate` *tabla con el estado del documento activo establecido en “Mock Doc”*.
- **Inyección ambiental:** *La ruta a esta base de datos simulada se pasa a todos los procesos secundarios (scripts de Python y el host MCP) a través de entorno variable.TESTDBFILE*

2. Agrupación de pruebas

La suite está dividida en varios bloques lógicos:

A. Página de inicio Cordura y visibilidad de la sección

Valida que la compilación de Eleventy sea correcta y que todas las secciones principales de la interfaz de usuario estén presentes.

- **Pruebas genéricas:** En lugar de buscar títulos de artículos específicos, verifica la existencia de clases estructurales como .hero-featured, .app-icon-card, y .post-card.
- **Resiliencia:** Las pruebas verifican que existe al menos uno elemento en cada cuadrícula, lo que garantiza que la prueba no falle solo porque se cambió el nombre o se eliminó un artículo específico.

B. Cambio de idioma y persistencia

- Prueba la lógica de internacionalización (i18n).
- Verifica que al hacer clic en los indicadores se actualice la URL (por ejemplo, /?lang=de).
- Valida que la preferencia lingüística persiste en localStorage y aplicado correctamente al recargar una página.

C. Navegación robusta (juegos y blog)

- Prueba la integridad del enlace en todo el sitio.
- **Enlace genérico siguiente:** Las pruebas encuentran dinámicamente el primer enlace de juego o blog disponible, hacen

- clic en el y verifican que la URL resultante sea correcta y que el cuerpo de la página se cargue correctamente.
- **Comprobación del encabezado:** No requiere estrictamente una en páginas de juegos, que admiten motores de juego personalizados que pueden utilizar un lienzo de pantalla completa.

D. API de herramientas de Writer Studio (integración con MCP)

Valida el servidor MCP (Model Context Protocol) basado en Python.

- **Proceso de desove:** Aparece pluginhost.py como un proceso infantil.
- **Comunicación JSON-RPC:** Se comunica con el anfitrión a través de stdin/stdout utilizando cargas útiles JSON-RPC estándar.

- **Cobertura de herramientas:** Ejerce el , y herramientas contra la base de datos simulada.searchdocumentsbatchoperationsreplacetext

E. Herramientas anticomplementos (ejecución directa)

- Prueba directamente los complementos independientes de Python utilizados para la gestión activa de documentos.
- Verifica eso , , y Interactuar correctamente con SQLite
- **Serialización:** Este bloque utiliza tabla.readactivedocument.pyeditactivedocument.pyreplacetest.describese.serial para evitar colisiones de escritura simultáneas en la base de datos simulada del trabajador.